Axona Quick Start

Get a working pub/sub roundtrip in **under five minutes**. Two Node peers, one process, no browser, no bridge. Once it works you've verified your setup; jump to the Programmer Guide for the full picture and the API Reference for every exported symbol.

Prerequisites

- **Node.js 20+** (for built-in Web Crypto Ed25519)
- A terminal

That's it. No build step, no DB, no bridge.

1. Install (30 seconds)

```
mkdir my-axona-demo && cd my-axona-demo
npm init -y
npm pkg set type=module
npm install github:axona-net/axona-protocol#v2.8.0
```

2. Write the demo (one file)

Save this as `index.js`:

```
import {
  AxonaPeer, AxonaDomain, NeuronNode, AxonManager, Synapse,
  SimNetwork, simTransport,
  deriveIdentity, geoCellId, clz264,
} from '@axona/protocol';

// Region: us-east (Virginia).  Both peers live here.
const REGION = { lat: 38.0, lng: -77.0 };

// Synthetic publisher whose top 8 bits = the region's S2 cell, so
// the derived topic id has matching S2 prefix → us-east peers naturally
// cluster as the axon set.
function regionSynthPublisher({ lat, lng }) {
  const s2 = geoCellId(lat, lng, 8);
  return s2.toString(16).padStart(2, '0') + '0'.repeat(64);
}

// Build a peer from kernel primitives.
async function makePeer(network) {
  const identity = await deriveIdentity(REGION);
  const transport = simTransport({ network, identity, heartbeatMs: 0 });
  await transport.start(identity.id);

  const node = new NeuronNode({
    id: BigInt('0x' + identity.id),     // NeuronNode XORs IDs as BigInts
    lat: REGION.lat, lng: REGION.lng,
```

```
  });
  node.transport = transport;
  const domain = new AxonaDomain({ k: 20 });
  const peer = new AxonaPeer({ domain, node, identity, transport });
  await peer.start();

  // Wire AxonManager (pub/sub layer) to AxonaPeer's DHT primitives.
  const dht = {
    getSelfId:       () => peer.getNodeId(),
    findKClosest:    (...a) => peer.findKClosest(...a),
    routeMessage:    (...a) => peer.routeMessage(...a),
    sendDirect: async (peerId, type, payload) => {
      if (peerId === peer.getNodeId()) {                    // self-loop
        const h = peer._directHandlers?.get(type);
        if (!h) return false;
        await h(payload, { fromId: peer.getNodeId(), type });
        return true;
      }
      return peer.sendDirect(peerId, type, payload);
    },
    onRoutedMessage: (t, h) => peer.onRoutedMessage(t, h),
    onDirectMessage: (t, h) => peer.onDirectMessage(t, h),
  };
  peer._axonManager = new AxonManager({ dht });
  return { peer, identity };
}

// Two peers on a shared SimNetwork.
const network = new SimNetwork();
const alice = await makePeer(network);
const bob   = await makePeer(network);

// Open a channel between them and admit each as a synapse (real
// transports do this via the axona:hello handshake on channel-open).
await alice.peer._transport.openConnection(bob.identity.id);
const admit = (local, remoteHex) => {
  const remote = BigInt('0x' + remoteHex);
  const syn = new Synapse({
    peerId: remote, latencyMs: 1,
    stratum: clz264(local._node.id ^ remote),
  });
  syn.weight = 0.5; syn.inertia = 0; syn._addedBy = 'demo';
  local._node.synaptome.set(remote, syn);
};
admit(alice.peer, bob.identity.id);
admit(bob.peer,   alice.identity.id);

// Pub/sub roundtrip.
const TOPIC     = 'us-east/hello-world';
const publisher = regionSynthPublisher(REGION);

const sub = await bob.peer.sub(TOPIC, (env) => {
  console.log('[bob]   received:', env.message);
}, { publisher, since: 'all' });

await new Promise(r => setTimeout(r, 100));   // let subscribe-k land

const msgId = await alice.peer.pub(TOPIC, 'hello from alice', { publisher });
console.log('[alice] published msgId=' + msgId);

await new Promise(r => setTimeout(r, 200));   // let fan-out complete
await sub.stop();
process.exit(0);
```

3. Run

```
node index.js
```

You should see:

```
[alice] published msgId=8e9d4b1a…
[bob]   received: hello from alice
```

That's it. You just ran:

- Ed25519 identity derivation (264-bit nodeIds, S2-cell-prefixed for us-east)
- A signed envelope publish through `AxonManager`'s K-closest replication
- A `since: 'all'` subscriber receiving the cached message

The full version of this with comments and a verifier is at `examples/minimal-pubsub/` — same code, ready to run.

What just happened

```
alice                                  bob
■■■■■                                   ■■■■■
peer.pub(TOPIC, …)                     peer.sub(TOPIC, handler, since:'all')
    ■                                      ■
    ▼                                      ▼
AxonManager                            AxonManager
  ■■ derive 264-bit topic id            ■■ derive same topic id
  ■   from the synth publisher          ■   (same publisher arg → same id)
  ■■ build signed envelope              ■■ send 'subscribe-k' to K-closest
  ■■ send 'publish-k' to K-closest      ■     (alice + bob — small mesh)
  ▼                                       ▼
alice cache role        ■■■■■■■■■■■   bob cache role
                                        (lazy-promoted on first publish-k)
                                        bob is a child of its own role
                                             ■
                                             ▼
                                        deliveryCallback(env) ■■ handler
```

Both peers compute identical 264-bit topic ids because they use the same `opts.publisher` value (the synthetic region id). That ID-matching is the **one rule you can't break** — see Pitfall #13.1 in the Programmer Guide.

Next steps

Want to…	Read
Understand the mental model	Programmer Guide §3
Build a chat / forum / feed app	Programmer Guide §12
Hook up a real WebRTC + bridge stack	`axona-peer/src/axona_node.js` — the reference browser wiring

Look up a specific function	API Reference
Run a bridge locally	Programmer Guide §11

Troubleshooting

`Cannot find module '@axona/protocol'` — make sure `package.json` has `"type": "module"` and the install completed. Re-run `npm install`.

`Cannot mix BigInt and other types` — your `NeuronNode` got an `id` of the wrong type. Pass `BigInt('0x' + identity.id)`, not the raw hex string.

Demo runs but nothing arrives at `bob` — the synaptome admission step is missing. Real transports admit peers via `axona:hello`; in this in-process demo you have to do it manually (the `admit(...)` calls above).

`UPGRADE_REQUIRED` **close code (4426)** — only happens when connecting to a bridge; not relevant for the in-process demo. If you see it when expanding to a bridge connection, your peer is older than the bridge's `MIN_PEER_VERSION`. Update both to v2.8.0+.